

Kolokwium 1 - Java

Imię:

Nazwisko:

Na napisanie testu jest 45 minut. Każde poprawnie rozwiązane zadanie to jeden punkt. Każde niepoprawnie rozwiązane zadanie to zero punktów (dotyczy to również zadań z wielokrotnym wyborem).

1. Poniższy kod:

```
class MyException1 extends RuntimeException {}
class MyException2 extends RuntimeException {}
public class Exceptional {
    public static void main(String [] args) {
        try { if (true) throw new MyException1(); }
        catch(MyException1 e) {
            System.out.print("E"); throw new MyException2();
        }
        catch(MyException2 e) {
            System.out.print("M"); throw new MyException1();
        }
    }
}
```

- a. zapętlili się (będzie wypisywał "EMEMEMEME...")
- b. wypisze "EM" i zakończy się z błędem
- c. wypisze "E"
- d. nie skompiluje się, bo main nie ma klauzuli throws.

2. Co wypisze poniższy kod?

```
public class SomeOrder {
    { System.out.print('a'); }
    static { System.out.print('b'); }
    SomeOrder() { System.out.print('c'); }
    static { System.out.print('d'); }
    { System.out.print('e'); }
    public static void main(String [] args) {
        SomeOrder a = new SomeOrder();
    }
}
```

- a. cae
- b. aecbd
- c. abcde
- d. bdaec
- e. bdcae

3. Co zostanie wypisane po uruchomieniu poniższego kodu?

```
class StaticInnerPair {
```

```

static class Pair{
    int x, y;
    public Pair(int x, int y){
        this.x = x;
        this.y = y;
    }
}
public static Pair f(){
    Pair x = new Pair(8, 22);
    try {
        return x;
    } finally{
        x.x = 11;
        return x;
    }
}
public static void main(String [] args) {
    System.out.println(f().x + f().y + " " + f().x);
}
}

```

- a. 822 8
 - b. 1122 11
 - c. 30 11
 - d. 33 11
 - e. 33 8
4. Wybierz które z podanych poleceń można wstawić w miejsce ### aby kod się kompilował.

```

public interface Doable {
    public void doit();
}
public class WhatToDo implements Doable {
    public void doit() {
        //...
    }
    public void createAnonymousClass() {
        new Doable() {
            public void doit() {
                ###
            }
        };
    }
}

```

```
}  
}
```

- a. `WhatToDo.doit();`
- b. `this.doit();`
- c. `super.doit();`
- d. `WhatToDo.this.doit();`
- e. `Doable.doit();`
- f. `Doable.this.doit();`
- g. `WhatToDo.super.doit();`

5. Napisz co powoduje umieszczenie pola `final` w

a. definicji pola:

b. definicji metody:

c. definicji klasy:

6. Jeśli klasa A dziedziczy po `Object` i deklaruje jedną metodę

```
public Object t(String s) { return "A"; }
```

to klasa B dziedzicząca po A może posiadać metodę:

- a. `public String t(Object o) { return "B"; }`
 - b. `String t(String s) { return "B"; }`
 - c. `public Object t(String s) { return "B"; }`
 - d. `public String t(String s) { return "B"; }`
 - e. `Integer t(String s) { return 2; }`
 - f. `public Integer t(String s) { return 2; }`
 - g. `public Integer t(Integer i) { return 2; }`
7. Poniższy kod kompiluje się i uruchamia bez problemów. Jaki jest wynik jego wykonania?

```
class A{}  
class B extends A{}  
class C extends B{}  
public class Polimorfizm {  
    static void mimi(A x){System.out.print("A");}  
    static void mimi(B x){System.out.print("B");}  
    static void mimi(C x){System.out.print("C");}  
    public static void main(String[] args) {  
        B x = new C();  
        mimi(x);  
        mimi(null);  
    }  
}
```

```
}
```

8. W którym miejscu poniższej metody obiekt zainicjowany w linii 1 może zostać (najwcześniej !) odśmiecony:

```
void method() {  
    String a = new String("aaa"); // 1  
    a = a+1; //2  
    a = null; //3  
} //4
```

- a. przed wykonaniem 2
 - b. przed wykonaniem 3
 - c. przed wykonaniem 4
 - d. nigdy
9. Które z poniższych instrukcji wstawione w miejsce ### utworzą poprawnie instancję klasy Inner:

```
public class Outer {  
    class Inner {}  
    public static void main (String [] args) {  
        Outer a = new Outer();  
        ###  
    }  
}
```

- a. Outer.Inner b = new Outer.Inner();
 - b. Outer.Inner b = a.new Inner();
 - c. Inner b = new a.Inner();
 - d. Inner b = a.new Inner();
10. Kiedy String "Ala" może być najwcześniej odśmiecony?

```
class Stack {  
    private String[] stack;  
    private int size = 0;  
    public Stack put(String s) {  
        String[] old = stack;  
        stack = new String[size + 1];  
        for(int i = 0; i<size; i++) stack[i]=old[i];  
        stack[size++]=s;  
        return this;  
    }  
    public String pop(){  
        return stack[--size];  
    }  
    public static void main(String[] args) {  
        Stack s = new Stack(); //1
```

```

        s.put("Ala").put("ma").put("kota").put("!"); //2
        System.out.println(s.pop() + s.pop() + s.pop() +
s.pop()); //3
        s.put("Kot").put("ma").put("Ale").put("."); //4
    } //5
}

```

- a. przed wykonaniem 2
- b. przed wykonaniem 3
- c. przed wykonaniem 4
- d. przed wykonaniem 5
- e. nigdy

11. Zaznacz wszystkie poprawne deklaracje metody main.

- a. `public static void main(String[] args) {}`
- b. `public void main(String[] args) {}`
- c. `public static void main() {}`
- d. `public static void main(String... args) {}`
- e. `static void main(String[] args) {}`

12. Które z poniższych deklaracji niezagnieżdżonej klasy A są niepoprawne?

- a. `public final class A { }`
- b. `abstract final class A { }`
- c. `private final class A { }`
- d. `abstract protected class A { }`
- e. `class A { }`
- f. `final class A { }`
- g. `public static class A { }`
- h. `abstract class A { }`

Zanalizuj poniższe kody (zakładamy że wszystkie importy są w porządku) i:

- jeśli nastąpi błąd kompilacji klasy, to wskaż w którym miejscu,
- jeśli klasa się skompiluje, ale po uruchomieniu wyrzucony zostanie wyjątek to wskaż w której linii,
- jeśli kod się wykona bez wyjątku to podaj wynik uruchomienia klasy posiadającej main.

13.

```

interface A {
    A test();
}

```

```

interface B {
    B test();
}

interface C extends A, B {
    C test();
}

public class Zadanie {
    public static void main(String args[]) {
        System.out.println(new C() {
            @Override
            public C test() {
                return this;
            }
        }.test() == null);
    }
}

```

14.

```

class Test {
    final int i;
    Object o = new Object(){
        @Override
        public String toString(){
            return String.valueOf(i);
        }
    };
    Test(int i) {
        this.i = i;
    }
    public static void main(String[] args) {
        System.out.println(new Test(11).o);
    }
}

```

15.

```

class A {
    int i;
}
class B extends A {

```

```
    A parent = this;
    B(int i) { parent.i = i;}
    @Override
    public String toString(){
        return "numer " + (parent.getClass() == getClass() ?
this.i: parent.i + 1);
    }
}
class Test{
    public static void main(String[] args){
        System.out.println(new B(17));
    }
}
```